Student-Friendly Pseudocode Guide

Contents

Student-Friendly Pseudocode Guide 1

Cambridge International AS & A Level Computer Science 9618 3

1. Reading and writing pseudocode 3

2. Variables, constants and data types 3

Common data types 3

Declaring and assigning 4

Constants 4

3. Arrays 4

4. User-defined data types 5

Enumerated types 5

Records 5

Sets and pointers 5

5. Common operations 6

Input and output 6

Arithmetic 6

Comparisons and logic 6

String operations 7

Numeric functions 7

6. Selection: choosing what happens 7

IF statements 7

CASE statements 8

7. Loops: repeating instructions 8

FOR loops 8

WHILE loops 9

REPEAT loops 9

8. Procedures and functions 9

Procedures 9

Functions 10

Passing parameters 10

9. File handling 10

Text files 10

Random files 11

10. Object-oriented programming 11

Exam-ready habits 12

Cambridge International AS & A Level Computer Science 9618

This guide introduces the pseudocode conventions used in Cambridge exams for 2027–2029. Pseudocode is a way to describe an algorithm clearly without tying it to one programming language. Use these conventions when you practise, and read each question carefully: an exam may explain additional rules or functions.

1. Reading and writing pseudocode

- **Keywords** are usually written in UPPERCASE, such as `IF`, `WHILE` and `PROCEDURE`.
- **Names** for variables and other identifiers use mixed case, such as `PlayerScore` or `TotalToPay`.
- **Indentation** shows which statements belong inside a selection or loop. Keep the layout neat.
- **Comments** begin with `//`. They explain the code but are not instructions for the computer.
- **Line numbers** are labels for referring to lines; they are not part of the pseudocode.
- An angle-bracket phrase such as `<condition>` is a placeholder. Replace it with the actual condition or statement.

```
// Increase the score after a correct answer
Score ← Score + 1
```

Tip: Indent the statements inside `IF`, loops, procedures and functions. Good layout makes logic easier to trace and mistakes easier to spot.

2. Variables, constants and data types

A **variable** stores a value that can change while a program runs. A **constant** stores a value that should stay the same.

Common data types

Type	What it stores	Example
`INTEGER`	A whole number	`12`, `-3`
`REAL`	A number that may have a fractional part	`4.7`, `-4.0`
`CHAR`	One character in single quotes	`'A'`
`STRING`	Text in double quotes; it can be empty	`"Hello"`, `""`
`BOOLEAN`	One of two logical values	`TRUE`, `FALSE`
`DATE`	A calendar date	`02/01/2025`

An identifier must start with a letter and may contain letters, digits and underscores. Choose a name that tells you what the value represents. Avoid using a pseudocode keyword as a name.

Declaring and assigning

`DECLARE` gives a variable a name and a data type. The arrow `←` assigns a value to it.

```
DECLARE Score : INTEGER
DECLARE PlayerName : STRING
DECLARE GameOver : BOOLEAN

Score ← 0
PlayerName ← "Ari"
GameOver ← FALSE
Score ← Score + 1
```

The value assigned must be suitable for the variable's data type. For example, text should not be assigned to an `INTEGER`.

Constants

Use a constant for a fixed value that has a meaningful name.

```
CONSTANT MaximumPlayers = 4
CONSTANT WelcomeText = "Ready to play?"
```

A constant's value is written as a literal value, such as a number or a string.

3. Arrays

An **array** stores several values of the same data type under one name. Each item is found using its **index**. State the index range clearly; in Cambridge examples, the first index is often `1`.

```
DECLARE TeamNames : ARRAY[1:4] OF STRING

TeamNames[1] ← "Ari"
TeamNames[2] ← "Mina"
```

A two-dimensional array uses two indexes, like rows and columns on a grid.

```
DECLARE Board : ARRAY[1:3,1:3] OF CHAR
Board[2,3] ← 'X'
```

Use a loop to work through or update many array elements:

```
FOR Index ← 1 TO 4
    OUTPUT TeamNames[Index]
NEXT Index
```

Tip: Check that every index is within the array's declared range. An index that is too small or too large can cause an error.

4. User-defined data types

A **user-defined type** lets you create a data type that better fits your problem.

Enumerated types

An enumerated type lists the allowed choices. It is useful when a value should be selected from a small, known set.

```
TYPE Season = (Spring, Summer, Autumn, Winter)

DECLARE CurrentSeason : Season
CurrentSeason ← Summer
```

Records

A **record** groups related pieces of information, which may have different data types. Think of it like one student profile containing several fields.

```
TYPE StudentRecord
    DECLARE FirstName : STRING
    DECLARE YearGroup : INTEGER
    DECLARE FormGroup : CHAR
ENDTYPE

DECLARE Pupil : StudentRecord
Pupil.FirstName ← "Leroy"
Pupil.YearGroup ← 12
Pupil.FormGroup ← 'A'
```

Use a dot to access a record's individual fields, as in `Pupil.FirstName`.

Sets and pointers

A **set** represents a collection of values of the same type. A **pointer** refers to a location in memory. These types appear in the syllabus; follow the definitions and rules provided in your course and exam questions.

```
TYPE LetterSet = SET OF CHAR
TYPE IntegerPointer = ^INTEGER

DEFINE Vowels ('A','E','I','O','U') : LetterSet
DECLARE MyPointer : IntegerPointer
```

Tip: Before using a user-defined type, define it. Then declare variables using that type.

5. Common operations

Input and output

`INPUT` gets a value from the user. `OUTPUT` displays a value or message.

```
INPUT PlayerName
OUTPUT "Welcome, ", PlayerName
```

You can output several items in one statement by separating them with commas.

Arithmetic

Operator	Meaning
`+`	Add
`-`	Subtract
`*`	Multiply
`/`	Divide; the result is a `REAL`
`DIV`	Integer division; gives the whole-number quotient
`MOD`	Gives the remainder after division

For example, integer division and remainder can help split items into equal groups. Use parentheses when an expression might otherwise be unclear.

```
WholeGroups ← NumberOfItems DIV GroupSize
LeftOver ← NumberOfItems MOD GroupSize
```

Comparisons and logic

Comparisons produce a Boolean value: `TRUE` or `FALSE`.

Operator	Meaning
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal to
`<=`	Less than or equal to
`=`	Equal to

Operator	Meaning
`<>`	Not equal to

Use `AND`, `OR` and `NOT` to combine or change Boolean conditions.

```
IF Age >= 16 AND HasPermission = TRUE THEN
    OUTPUT "You may continue"
ENDIF
```

Tip: Use parentheses in a complicated condition to show exactly which comparisons belong together.

String operations

Strings are text. The operator `&` joins strings together, a process called **concatenation**.

```
FullName ← FirstName & " " & LastName
```

Common string functions include:

Function	What it does
`LENGTH(Text)`	Returns the number of characters
`RIGHT(Text, Count)`	Returns the rightmost characters
`MID(Text, Start, Count)`	Returns a section of a string
`LCASE(Character)`	Changes an uppercase character to lowercase
`UCASE(Character)`	Changes a lowercase character to uppercase

For example, `RIGHT("ABCDEFGH", 3)` returns `"FGH"`, and `MID("ABCDEFGH", 2, 3)` returns `"BCD"`.

Numeric functions

- `INT(Value)` returns the integer part of a real number. For example, `INT(27.5415)` returns `27`.
- `RAND(Maximum)` returns a random real number from `0` up to, but not including, the maximum.

Check the question for any additional functions: exam questions will define pseudocode functions that they expect you to use.

6. Selection: choosing what happens

IF statements

Use `IF` when the program needs to make a decision. The optional `ELSE` gives an alternative path.

```
IF Score >= 50 THEN
    OUTPUT "Pass"
ELSE
    OUTPUT "Keep practising"
ENDIF
```

An `IF` can also be used without an `ELSE`:

```
IF Answer = "Yes" THEN
    OUTPUT "Thanks for confirming"
ENDIF
```

You can place one `IF` inside another when a decision depends on more than one step. Keep each level indented so the structure is clear.

CASE statements

Use `CASE OF` when one variable may have several possible values. `OTHERWISE` is an optional final branch for any value not listed.

```
INPUT Choice

CASE OF Choice
    'A' : OUTPUT "Add a player"
    'R' : OUTPUT "Remove a player"
    'Q' : OUTPUT "Quit"
    OTHERWISE : OUTPUT "Choose A, R or Q"
ENDCASE
```

Once a matching case is found, its statements run and the program continues after `ENDCASE`.

Tip: Use `IF` for conditions such as “score is greater than a limit.” Use `CASE` when comparing one value with several specific choices.

7. Loops: repeating instructions

FOR loops

Use a `FOR` loop when you know how many times to repeat an action. The start and end values are included.

```
FOR Count ← 1 TO 5
    OUTPUT Count
NEXT Count
```

`STEP` changes how the loop counter moves. It can also be negative.

```
FOR Count ← 10 TO 2 STEP -2
    OUTPUT Count
NEXT Count
```

If the starting value is greater than the ending value and the step does not move toward it, the loop body will not run.

WHILE loops

A `WHILE` loop checks its condition **before** each repetition. It may run zero times if the condition is false at the start.

```
WHILE Lives > 0
    OUTPUT "Keep playing"
    Lives ← Lives - 1
ENDWHILE
```

Make sure something inside the loop can change the condition. Otherwise, the loop may never end.

REPEAT loops

A `REPEAT` loop checks its condition **after** running the loop body, so the body always runs at least once.

```
REPEAT
    OUTPUT "Enter a password"
    INPUT Password
UNTIL Password = "Secret"
```

Quick check: `FOR` is usually for a known number of repetitions; `WHILE` checks first; `REPEAT` checks afterward.

8. Procedures and functions

Procedures and functions let you give a useful name to a group of instructions. This helps keep a program organised and makes code easier to reuse.

Procedures

A **procedure** performs an action. Use `CALL` to run it.

```
PROCEDURE ShowWelcome(Name : STRING)
    OUTPUT "Welcome, ", Name
ENDPROCEDURE

CALL ShowWelcome("Ari")
```

The values supplied when calling a procedure are **arguments**. The named inputs in the procedure definition are **parameters**. Their data types and order should match.

Functions

A **function** calculates and returns one value. Use `RETURN` to send that value back. A function call is part of an expression, so do not put `CALL` before it.

```
FUNCTION Larger(Number1 : INTEGER, Number2 : INTEGER) RETURNS INTEGER
    IF Number1 > Number2 THEN
        RETURN Number1
    ELSE
        RETURN Number2
    ENDIF
ENDFUNCTION

OUTPUT Larger(8, 5)
```

Passing parameters

By default, parameters are passed **by value**: the procedure or function receives a copy of the value. A parameter passed **by reference** can change the original variable. The keywords `BYVAL` and `BYREF` can specify the method.

```
PROCEDURE IncreaseScore(BYREF Score : INTEGER)
    Score ← Score + 1
ENDPROCEDURE
```

Use `BYREF` only when the original value needs to be changed. Parameters should not be passed by reference to a function.

Tip: A function returns a value; a procedure performs an action. This is a helpful way to decide which one to use.

9. File handling

Files let a program keep data after it finishes running. A file must be opened before you read or write it, and closed when you have finished.

Text files

Text files contain lines of text. Open a text file using one of these modes:

- `READ` to get data from a file.
- `WRITE` to create or replace a file. Existing contents are lost.
- `APPEND` to add data after existing contents.

`READFILE` reads the next line into a string variable. `WRITEFILE` writes data to a file. `EOF` is `TRUE` when there are no more lines to read.

```
DECLARE LineOfText : STRING

OPENFILE "Notes.txt" FOR READ

WHILE NOT EOF("Notes.txt")
    READFILE "Notes.txt", LineOfText
    OUTPUT LineOfText
ENDWHILE

CLOSEFILE "Notes.txt"
```

Random files

A random file stores records that can be accessed directly by their location. Open it using `RANDOM`, use `SEEK` to choose a record location, and then use `GETRECORD` or `PUTRECORD` to read or write a record.

```
OPENFILE "StudentFile.dat" FOR RANDOM
SEEK "StudentFile.dat", 3
GETRECORD "StudentFile.dat", Pupil
CLOSEFILE "StudentFile.dat"
```

`PUTRECORD` replaces the record at the current location. Be careful to calculate and explain file addresses clearly when a question asks you to.

Tip: Match each operation to the file mode. Close every file once you have finished using it.

10. Object-oriented programming

Object-oriented programming groups data and actions together.

- A **class** is a plan or template.
- An **object** is one instance made from a class.
- A **property** stores information about an object.
- A **method** is an action the object can perform.
- A **constructor** sets up a new object. In this pseudocode, it is a procedure named `NEW`.
- **Inheritance** lets one class build on another. `INHERITS` identifies the parent class, and `SUPER` can call a parent-class method.

```
CLASS Pet
    PRIVATE Name : STRING

    PUBLIC PROCEDURE NEW(GivenName : STRING)
        Name ← GivenName
    ENDPROCEDURE

    PUBLIC PROCEDURE SetName(GivenName : STRING)
```

```
        Name ← GivenName
    ENDPROCEDURE
ENDCLASS

MyPet ← NEW Pet("Milo")
MyPet.SetName("Pip")
```

`PUBLIC` means the property or method can be accessed from outside the class. `PRIVATE` means it is intended for use within the class. If access is not specified, exam pseudocode may treat methods and properties as public.

Tip: Think of a class as a recipe and an object as one item made from that recipe. Each object can have its own property values.

Exam-ready habits

- Read the question’s definitions and conventions before writing code.
- Declare variables with suitable data types and choose descriptive names.
- Check array indexes and loop start, end and step values.
- Trace selection conditions carefully, including `AND`, `OR` and `NOT`.
- Make sure loops can end.
- Use a procedure for an action and a function when you need a returned value.
- Indent consistently and add short comments when they clarify your algorithm.